Phase 2: Database Schema

AI Support & Knowledge Ops Platform — explained in plain language

---

What is Phase 2?

Phase 2 builds the database — basically the set of filing cabinets where the whole project stores its information. Before any AI feature can work (answering questions, creating support tickets, remembering a chat), there needs to be a place to put that data. That's what this phase creates: a Neon Postgres database with 5 tables, using a special extension called **pgvector** that lets the database search text by meaning, not just by matching words.

A quick analogy

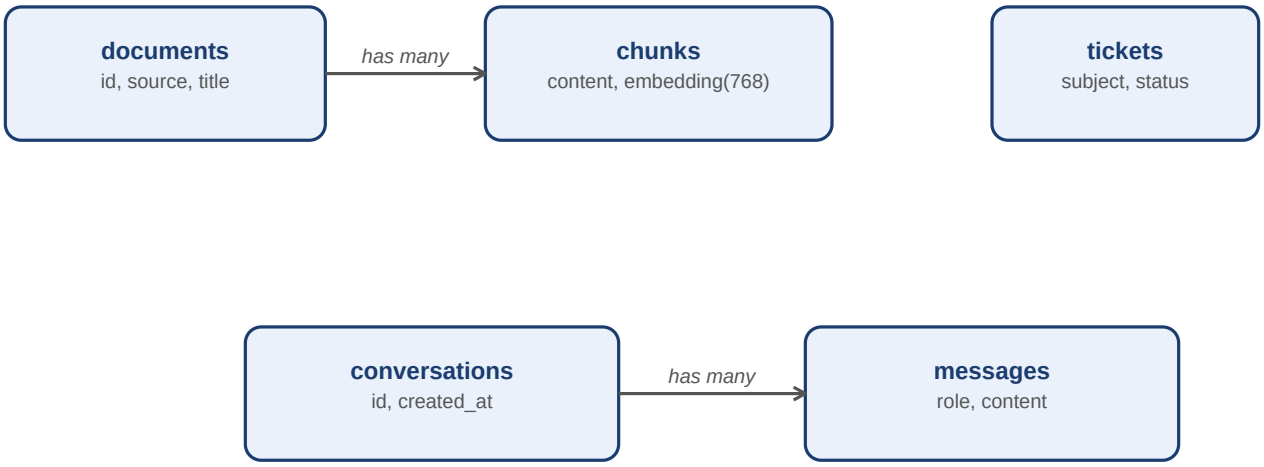
Think of a library. Each book is a **document**. Librarians don't memorize every word in every book — instead they tear out the most useful pages and summarize them on index cards, which are the **chunks**. Each index card also gets a little scent attached to it that represents what the card is about (that scent is the **embedding**) — two cards about similar topics will smell similar, even if they don't use the same words. When someone asks a question, the librarian doesn't search word-by-word; they sniff for cards with a similar scent to the question, pull the best few, and answer using those. That's exactly how this database lets the AI answer support questions.

The 5 tables, in simple terms

Table	What it stores (in plain words)	Used later for
documents	One entry per help article the AI has read (title + where it came from)	Showing the user "this answer came from this article"
chunks	Each article broken into small pieces of text, with a list of numbers attached to each piece that captures its meaning	Letting the AI search by meaning to find the most relevant piece of text for a question
tickets	Fake support tickets (subject, description, status like "open")	The mock helpdesk system — the AI agent creates and checks these
conversations	One entry per chat session	Grouping messages that belong together
messages	Each message in a chat, who said it (user/AI), and what it said	Remembering chat history

How the 5 tables connect (diagram)

The arrows below mean "one of these can have many of those." One document is cut into many chunks. One conversation can hold many messages. Tickets stand alone — they don't link to anything else yet.

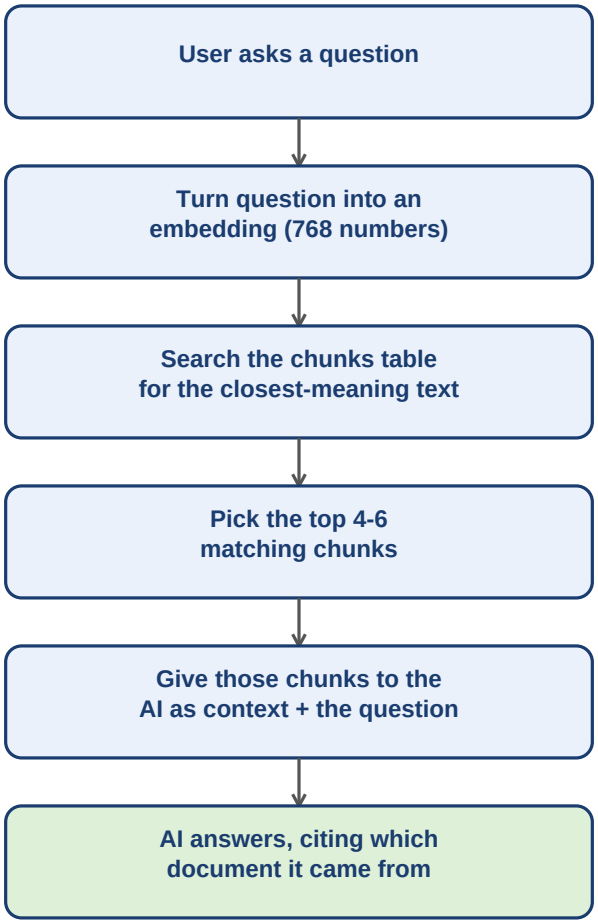


Why "chunks" and the special index matter

Articles are too long to hand to the AI all at once, so each one is cut into small pieces ("chunks") of about 500 characters. Every chunk also gets an **embedding** — a list of 768 numbers that represents what that piece of text means. When a user asks a question, the system turns the question into numbers the same way, then finds the chunks whose numbers are closest to it — that's how the AI finds relevant information by meaning instead of exact keyword matching. A special index (HNSW) is added on top so this search stays fast even as more documents are added.

What happens when a user asks a question (flowchart)

This is the journey a question takes later, in Phase 6, once this database exists. It shows exactly why the **chunks** table was built the way it was.



What was actually built

- Turned on the pgvector extension in Postgres (the "meaning search" feature).
- Created the **documents** table.
- Created the **chunks** table, including the fast-search index.
- Created the **tickets** table for the mock support desk.
- Created the **conversations** and **messages** tables for chat history.

Where each piece gets used later in the project

- **documents / chunks** → filled in Phase 3 (loading the knowledge base) and read in Phase 6, when the AI answers a question with sources.
- **tickets** → used in Phase 8, when the AI agent creates or checks a support ticket on request.

• **conversations / messages** → used in Phase 7, so the AI agent remembers what was said earlier in the same conversation.

Glossary (for beginners)

Database: A program that stores information in organized tables, so it can be saved, searched, and updated later instead of living only in memory.

Table: Like a spreadsheet: it has columns (fields) and rows (one entry per row).

Postgres: A popular, free, open-source type of database. Neon is a company that hosts Postgres for you on the internet so you don't run it on your own computer.

Extension (pgvector): An add-on that gives Postgres a new superpower — here, the power to store and search "meaning vectors" (embeddings).

Embedding: A list of numbers that represents the meaning of a piece of text. Similar meanings produce similar number lists, even with completely different words.

Vector search / similarity search: Instead of searching for exact words, the database finds entries whose embedding numbers are mathematically closest to the question's embedding — i.e., closest in meaning.

Index (HNSW): A shortcut structure that lets the database find the closest matches fast, instead of comparing the question to every single chunk one by one.

Foreign key: A column that points to a row in another table — e.g. every chunk points to the document it came from, via document_id.

Current status

The table-creation script (`scripts/init_db.sql`) is written and matches everything above. What hasn't been independently confirmed yet is whether this script has actually been run against the real Neon database and checked with a test insert — that last verification step is still open.